Clase 04: Servicios Web, API REST, JSON y Bases de Datos NoSQL

Semana 4 | Corte 1

Prof. Jefferson Sarmiento Rojas

2026-08-19

Table of contents

Objetivos de la Clase 1
Servicios Web y Protocolo HTTP 2
El Protocolo HTTP 2
API REST 2
Bases de Datos No Relacionales (NoSQL) y JSON 3
Operaciones CRUD 3
Código de Referencia: Login y CRUD con Firebase NoSQL 3
1. Estructura HTML (`index.html`) 3
2. Estilos CSS (`style.css`) 6
3. Lógica Principal JavaScript (`script.js`) 10
4. Módulo de Conexión a Firebase (`api.js`) 15
Materiales y Lecturas 17
Práctica de Laboratorio 17

Ver Presentación de Diapositivas

Objetivos de la Clase

- ☐ Comprender el concepto de Servicios Web y su arquitectura basada en Request/Response.
- ☐ Conocer los métodos fundamentales del Protocolo HTTP (GET, POST, PUT, DELETE).
- ☐ Entender qué es una API REST y cómo permite la comunicación entre sistemas.
- ☐ Explorar el modelo de Bases de Datos No Relacionales (NoSQL) y el formato JSON.
- ☐ Aplicar operaciones CRUD en una base de datos NoSQL utilizando Firebase.

Servicios Web y Protocolo HTTP

Un **Servicio Web** es un componente de software que utiliza protocolos y estándares para permitir el intercambio de datos a través de Internet entre distintas aplicaciones. Se basan en el paradigma cliente-servidor mediante ciclos de **Request** (Petición) y **Response** (Respuesta).

El Protocolo HTTP

HTTP (Hypertext Transfer Protocol) es el protocolo estándar para la transferencia de información en la web. Trabaja mediante los siguientes métodos principales:

- **GET:** Consultar y obtener recursos.
- **POST:** Crear nuevos recursos.
- **PUT:** Modificar o actualizar recursos existentes.
- **DELETE:** Eliminar recursos.

Códigos de Respuesta HTTP

Cuando un servidor responde a una petición HTTP, incluye un código de estado para indicar el resultado: * **200 OK:** La petición se realizó con éxito. * **201 Created:** El recurso se creó correctamente (típico en POST). * **400 Bad Request:** La petición del cliente tiene errores. * **404 Not Found:** El recurso solicitado no existe. * **500 Internal Server Error:** Ocurrió un error en el servidor.

API REST

REST (Representational State Transfer) es una interfaz o estilo de arquitectura para el diseño de servicios web. Una API RESTful utiliza HTTP de forma eficiente para que un cliente pueda obtener, crear, modificar o eliminar datos (operaciones CRUD) alojados en un servidor, de forma rápida e independiente del lenguaje de programación que utilice cada sistema.

Bases de Datos No Relacionales (NoSQL) y JSON

Las bases de datos No Relacionales (NoSQL) no utilizan el esquema clásico de tablas estructuradas (como MySQL). En su lugar, utilizan modelos flexibles. Uno de los modelos más populares es el de **Bases de Datos Orientadas a Documentos**.

En estos sistemas, los datos suelen almacenarse y transmitirse utilizando el formato **JSON** (JavaScript Object Notation), el cual es altamente legible tanto para humanos como para máquinas.

Operaciones CRUD

Las bases de datos NoSQL también aplican el patrón **CRUD** fundamental para la gestión de la información: * **C**reate (Crear) * **R**ead (Leer / Consultar) * **U**pdate (Actualizar) * **D**elete (Eliminar)

Código de Referencia: Login y CRUD con Firebase NoSQL

A continuación se presentan los archivos separados necesarios para construir la práctica de la clase utilizando Firebase Realtime Database (sistema NoSQL basado en JSON).

1. Estructura HTML (`index.html`)

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Login, Registro y CRUD - Firebase NoSQL</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <div class="container">
        <!-- Login -->
        <div class="card" id="login-container">
            <h1>Iniciar Sesión</h1>
            <form id="login-form">
                <div class="form-group">
```

```
                <label for="login-email">Email</label>
                <input type="email" id="login-email" placeholder="correo@ejemplo.com" rec
            </div>
            <div class="form-group">
                <label for="login-password">Contraseña</label>
                <input type="password" id="login-password" placeholder="Contraseña" requ
            </div>
            <button type="submit">Ingresar</button>
        </form>
        <p class="switch">¿No tienes cuenta? <a href="#" id="show-register">Regístrate ac
        <div id="login-message"></div>
    </div>

    <!-- Registro -->
    <div class="card hidden" id="register-container">
        <h1>Registro</h1>
        <form id="register-form">
            <div class="form-group">
                <label for="register-name">Usuario</label>
                <input type="text" id="register-name" placeholder="Nombre de usuario" rec
            </div>
            <div class="form-group">
                <label for="register-email">Email</label>
                <input type="email" id="register-email" placeholder="correo@ejemplo.com"
            </div>
            <div class="form-group">
                <label for="register-password">Contraseña</label>
                <input type="password" id="register-password" placeholder="Contraseña" re
            </div>
            <button type="submit">Registrarse</button>
        </form>
        <p class="switch">¿Ya tienes cuenta? <a href="#" id="show-login">Inicia sesión</a
        <div id="register-message"></div>
    </div>

    <!-- Panel CRUD (solo tras iniciar sesión) -->
    <div id="panel" class="hidden">
        <div class="card">
            <h1>CRUD de Usuarios</h1>
            <div id="profile-info">
                <p>Sesión activa: <strong id="sesion-usuario"></strong></p>
                <button type="button" id="btn-logout" class="secundario">Cerrar sesión</b
```

```
            </div>

            <form id="usuario-form">
                <input type="hidden" id="usuario-id">
                <div class="form-group">
                    <label for="usuario">Usuario</label>
                    <input type="text" id="usuario" placeholder="Nombre de usuario" requ
                </div>
                <div class="form-group">
                    <label for="email">Email</label>
                    <input type="email" id="email" placeholder="correo@ejemplo.com" requ
                </div>
                <div class="form-group">
                    <label for="password">Contraseña</label>
                    <input type="text" id="password" placeholder="Contraseña" required m
                </div>
                <button type="submit" id="btn-guardar">Guardar</button>
                <button type="button" id="btn-cancelar" class="secundario hidden">Cancel
            </form>
            <div id="mensaje"></div>
        </div>

        <div class="card">
            <h1>Registros</h1>
            <table id="tabla">
                <thead>
                    <tr>
                        <th>Usuario</th>
                        <th>Email</th>
                        <th>Contraseña</th>
                        <th>Fecha</th>
                        <th>Acciones</th>
                    </tr>
                </thead>
                <tbody id="tabla-body"></tbody>
            </table>
            <p id="sin-datos" class="switch">Aún no hay registros.</p>
        </div>
    </div>
</div>

<script type="module" src="script.js"></script>
```

```
</body>
</html>
```

2. Estilos CSS (`style.css`)

```
* {
    box-sizing: border-box;
}

body {
    margin: 0;
    padding: 0;
    font-family: Arial, sans-serif;
    background: #f2f4f7;
    min-height: 100vh;
    display: flex;
    justify-content: center;
    align-items: flex-start;
    padding: 30px 0;
}

.container {
    width: 100%;
    max-width: 420px;
    padding: 20px;
    max-width: 640px;
}

.card {
    background: white;
    padding: 30px;
    border-radius: 12px;
    box-shadow: 0 5px 20px rgba(0, 0, 0, 0.1);
}

.card h1 {
    text-align: center;
    margin-bottom: 25px;
    color: #333;
}
```

```css
.form-group {
    margin-bottom: 18px;
}

.form-group label {
    display: block;
    margin-bottom: 7px;
    font-weight: bold;
    color: #444;
}

.form-group input {
    width: 100%;
    padding: 12px;
    border: 1px solid #ccc;
    border-radius: 6px;
    font-size: 15px;
}

.form-group input:focus {
    outline: none;
    border-color: #4285f4;
}

button {
    width: 100%;
    padding: 12px;
    border: none;
    border-radius: 6px;
    background: #4285f4;
    color: white;
    font-size: 16px;
    cursor: pointer;
}

button:hover {
    background: #3367d6;
}

button.secundario {
    margin-top: 10px;
    background: #6c757d;
```

```
}

button.secundario:hover {
    background: #545b62;
}

.switch {
    text-align: center;
    margin-top: 20px;
    color: #555;
}

.switch a {
    color: #4285f4;
    text-decoration: none;
    font-weight: bold;
}

.hidden {
    display: none;
}

.success {
    margin-top: 15px;
    padding: 10px;
    background: #d4edda;
    color: #155724;
    border-radius: 5px;
    text-align: center;
}

.error {
    margin-top: 15px;
    padding: 10px;
    background: #f8d7da;
    color: #721c24;
    border-radius: 5px;
    text-align: center;
}

#profile-info {
    background: #f5f5f5;
```

```
    padding: 15px;
    border-radius: 8px;
    margin-bottom: 20px;
}

#profile-info p {
    margin: 8px 0;
}

table {
    width: 100%;
    border-collapse: collapse;
}

th, td {
    padding: 10px;
    text-align: left;
    border-bottom: 1px solid #e3e6ea;
    font-size: 14px;
    word-break: break-all;
}

th {
    color: #444;
}

button.accion {
    width: auto;
    padding: 6px 12px;
    margin-right: 6px;
    font-size: 13px;
}

button.peligro {
    background: #dc3545;
}

button.peligro:hover {
    background: #b52b37;
}
```

3. Lógica Principal JavaScript (`script.js`)

```
import {
    registrarUsuario,
    iniciarSesion,
    crearUsuario,
    escucharUsuarios,
    actualizarUsuario,
    eliminarUsuario
} from './api.js';

document.addEventListener('DOMContentLoaded', () => {
    const loginContainer = document.getElementById('login-container');
    const registerContainer = document.getElementById('register-container');
    const panel = document.getElementById('panel');
    const showRegisterBtn = document.getElementById('show-register');
    const showLoginBtn = document.getElementById('show-login');
    const loginForm = document.getElementById('login-form');
    const registerForm = document.getElementById('register-form');
    const loginMessage = document.getElementById('login-message');
    const registerMessage = document.getElementById('register-message');
    const sesionUsuario = document.getElementById('sesion-usuario');
    const btnLogout = document.getElementById('btn-logout');

    const form = document.getElementById('usuario-form');
    const inputId = document.getElementById('usuario-id');
    const inputUsuario = document.getElementById('usuario');
    const inputEmail = document.getElementById('email');
    const inputPassword = document.getElementById('password');
    const btnGuardar = document.getElementById('btn-guardar');
    const btnCancelar = document.getElementById('btn-cancelar');
    const mensaje = document.getElementById('mensaje');
    const tablaBody = document.getElementById('tabla-body');
    const sinDatos = document.getElementById('sin-datos');

    let tablaIniciada = false;

    function mostrarMensaje(div, texto, tipo) {
        div.textContent = texto;
        div.className = tipo;
        setTimeout(() => {
            div.textContent = '';
```

```
            div.className = '';
        }, 3000);
    }

    function formatearFecha(timestamp) {
        if (!timestamp) return '-';
        return new Date(timestamp).toLocaleString('es-ES', {
            day: '2-digit',
            month: '2-digit',
            year: 'numeric',
            hour: '2-digit',
            minute: '2-digit'
        });
    }

    // Alternar vistas entre Login y Registro
    showRegisterBtn.addEventListener('click', (e) => {
        e.preventDefault();
        loginContainer.classList.add('hidden');
        registerContainer.classList.remove('hidden');
    });

    showLoginBtn.addEventListener('click', (e) => {
        e.preventDefault();
        registerContainer.classList.add('hidden');
        loginContainer.classList.remove('hidden');
    });

    function abrirPanel(sesion) {
        loginContainer.classList.add('hidden');
        registerContainer.classList.add('hidden');
        panel.classList.remove('hidden');
        sesionUsuario.textContent = `${sesion.usuario} (${sesion.email})`;
        if (!tablaIniciada) {
            iniciarTabla();
            tablaIniciada = true;
        }
    }

    btnLogout.addEventListener('click', () => {
        panel.classList.add('hidden');
        loginContainer.classList.remove('hidden');
```

```
    loginForm.reset();
});

// Registro
registerForm.addEventListener('submit', async (e) => {
    e.preventDefault();
    try {
        await registrarUsuario({
            usuario: document.getElementById('register-name').value.trim(),
            email: document.getElementById('register-email').value.trim(),
            password: document.getElementById('register-password').value
        });
        mostrarMensaje(registerMessage, 'Registro exitoso. Ya puedes iniciar sesión.', 's
        registerForm.reset();
        setTimeout(() => showLoginBtn.click(), 2000);
    } catch (error) {
        console.error(error);
        mostrarMensaje(registerMessage, error.message, 'error');
    }
});

// Login
loginForm.addEventListener('submit', async (e) => {
    e.preventDefault();
    try {
        const sesion = await iniciarSesion(
            document.getElementById('login-email').value.trim(),
            document.getElementById('login-password').value
        );
        abrirPanel(sesion);
    } catch (error) {
        console.error(error);
        mostrarMensaje(loginMessage, error.message, 'error');
    }
});

// --- CRUD ---
function salirDeEdicion() {
    form.reset();
    inputId.value = '';
    btnGuardar.textContent = 'Guardar';
    btnCancelar.classList.add('hidden');
```

```
    }

    form.addEventListener('submit', async (e) => {
        e.preventDefault();
        const datos = {
            usuario: inputUsuario.value.trim(),
            email: inputEmail.value.trim(),
            password: inputPassword.value
        };

        try {
            if (inputId.value) {
                await actualizarUsuario(inputId.value, datos);
                mostrarMensaje(mensaje, 'Registro actualizado.', 'success');
            } else {
                await crearUsuario(datos);
                mostrarMensaje(mensaje, 'Registro creado.', 'success');
            }
            salirDeEdicion();
        } catch (error) {
            console.error(error);
            mostrarMensaje(mensaje, 'Error al guardar: ' + error.message, 'error');
        }
    });

    btnCancelar.addEventListener('click', salirDeEdicion);

    // READ: se vuelve a pintar la tabla en cada cambio de la base
    function iniciarTabla() {
        escucharUsuarios((lista) => {
            tablaBody.textContent = '';
            sinDatos.classList.toggle('hidden', lista.length > 0);

            lista.forEach((registro) => {
                const fila = document.createElement('tr');

                const columnas = [
                    registro.usuario,
                    registro.email,
                    registro.password,
                    formatearFecha(registro.fechaCreacion)
                ];
```

```
            columnas.forEach((valor) => {
                const celda = document.createElement('td');
                celda.textContent = valor ?? '';
                fila.appendChild(celda);
            });

            const acciones = document.createElement('td');

            const btnEditar = document.createElement('button');
            btnEditar.type = 'button';
            btnEditar.className = 'accion';
            btnEditar.textContent = 'Editar';
            btnEditar.addEventListener('click', () => {
                inputId.value = registro.id;
                inputUsuario.value = registro.usuario ?? '';
                inputEmail.value = registro.email ?? '';
                inputPassword.value = registro.password ?? '';
                btnGuardar.textContent = 'Actualizar';
                btnCancelar.classList.remove('hidden');
                inputUsuario.focus();
            });

            const btnBorrar = document.createElement('button');
            btnBorrar.type = 'button';
            btnBorrar.className = 'accion peligro';
            btnBorrar.textContent = 'Eliminar';
            btnBorrar.addEventListener('click', async () => {
                if (!confirm(`¿Eliminar el registro "${registro.usuario}"?`)) return;
                try {
                    await eliminarUsuario(registro.id);
                    mostrarMensaje(mensaje, 'Registro eliminado.', 'success');
                    if (inputId.value === registro.id) salirDeEdicion();
                } catch (error) {
                    console.error(error);
                    mostrarMensaje(mensaje, 'Error al eliminar: ' + error.message, 'erro
                }
            });

            acciones.append(btnEditar, btnBorrar);
            fila.appendChild(acciones);
            tablaBody.appendChild(fila);
        });
```

```
        });
    }
});
```

4. Módulo de Conexión a Firebase (`api.js`)

```
// CRUD y autenticación simple sobre Firebase Realtime Database (NoSQL)
import { initializeApp } from "https://www.gstatic.com/firebasejs/10.8.1/firebase-app.js";
import {
    getDatabase,
    ref,
    push,
    set,
    update,
    remove,
    onValue,
    get,
    serverTimestamp
} from "https://www.gstatic.com/firebasejs/10.8.1/firebase-database.js";

const firebaseConfig = {
     apiKey: "AIzaSyA3GmMeLEhg8JtUWKsQZX9yPMsVmNEMioY",
  authDomain: "uriot-32f7d.firebaseapp.com",
  databaseURL: "https://uriot-32f7d.firebaseio.com",
  projectId: "uriot-32f7d",
  storageBucket: "uriot-32f7d.firebasestorage.app",
  messagingSenderId: "720901950215",
  appId: "1:720901950215:web:6b63c809e8b001dc6c53a2"
};

const app = initializeApp(firebaseConfig);
const db = getDatabase(app);
const usuariosRef = ref(db, "usuarios");

// Se filtra en el cliente para no depender de ".indexOn" en las reglas
async function buscarPorEmail(email) {
    const snapshot = await get(usuariosRef);
    if (!snapshot.exists()) return null;
    const encontrado = Object.entries(snapshot.val())
        .find(([, valores]) => valores.email === email);
```

```
    if (!encontrado) return null;
    return { id: encontrado[0], ...encontrado[1] };
}

// REGISTRO
export async function registrarUsuario({ usuario, email, password }) {
    if (await buscarPorEmail(email)) {
        throw new Error("Este correo ya está registrado.");
    }
    const nuevoRef = push(usuariosRef);
    await set(nuevoRef, { usuario, email, password, fechaCreacion: serverTimestamp() });
    return { id: nuevoRef.key, usuario, email };
}

// LOGIN
export async function iniciarSesion(email, password) {
    const encontrado = await buscarPorEmail(email);
    if (!encontrado || encontrado.password !== password) {
        throw new Error("Correo o contraseña incorrectos.");
    }
    return { id: encontrado.id, usuario: encontrado.usuario, email: encontrado.email };
}

// CREATE
export async function crearUsuario({ usuario, email, password }) {
    const nuevoRef = push(usuariosRef);
    await set(nuevoRef, { usuario, email, password, fechaCreacion: serverTimestamp() });
    return nuevoRef.key;
}

// READ: se dispara cada vez que cambian los datos en el servidor
export function escucharUsuarios(callback) {
    onValue(usuariosRef, (snapshot) => {
        const datos = snapshot.val() || {};
        const lista = Object.entries(datos).map(([id, valores]) => ({ id, ...valores }));
        callback(lista);
    });
}

// UPDATE
export async function actualizarUsuario(id, { usuario, email, password }) {
    await update(ref(db, `usuarios/${id}`), { usuario, email, password });
```

```
}

// DELETE
export async function eliminarUsuario(id) {
    await remove(ref(db, `usuarios/${id}`));
}
```

Materiales y Lecturas

> **Material de Clase**
>
> - Presentación: Sesión 6 (Documento PowerPoint)
> - Proyecto completo: Código fuente Firebase NoSQL (ZIP)

Práctica de Laboratorio

Actividad: Replicar el sistema de Login y CRUD consumiendo una base de datos NoSQL mediante Firebase y API REST, utilizando las tecnologías vistas en clase.